군-학 연계 프로그램 범용 교육

Claude Code로 바이브 코딩 — 내 홈페이지를 GitHub Pages에 올리기

서왕덕

경기대학교 AI컴퓨터공학부　8차시

26.11.11



목차

1. 복습과 로드맵
2. 1부 설치: Git · gh · Claude Code
3. 2부 Claude Code 기초와 GitHub 계정
4. 3부 팁: 막히지 않게, 막혔을 때
5. 4부 홈페이지 만들기와 배포
6. 정리



복습과 로드맵

말하기에서 만들기로

오늘 수업 로드맵 (120분)

시간	구간	내용	실습
8분	복습 · 로드맵	1 · 2차시 연결 · 바이브 코딩 · 오늘의 결과물	—
22분	**1부** 설치	Git · gh · Claude Code 설치, 로그인	실습 1
25분	**2부** 기초 · GitHub	권한 모드 · 명령 · 되돌리기 · GitHub 가입	실습 2 · 3 · 4
12분	**3부** 팁	요청법 · 막혔을 때 · CLAUDE.md · intent.md · 보안 · 사용량	—
45분	**4부** 홈페이지	레퍼런스 · 프롬프트 · 만들기 · 올리기 · Pages · 수정	실습 5 · 6 · 7
8분	정리	핵심 정리 · 집에서 이어하기 · PC 정리	—

◎ 오늘의 목표 하나

수업이 끝나면 각자 `https://아이디.github.io` 주소에 자기가 만든 홈페이지가 열린다. 코드는 한 줄도 직접 쓰지 않는다. 말로 시키고, 눈으로 확인하고, 다시 말한다. 실습 프롬프트 전문은 `prompts/` 폴더(00_README부터).

1 · 2차시에서 오늘로

1차시 프롬프트
잘 묻는 법. 5요소 (역할 · 맥락 · 과제 · 형식 · 제약). 답은 **내가 읽고 옮긴다**

2차시 스킬
잘 물은 것을 파일로 저장. 필요할 때 AI가 읽는다. 여전히 **결과는 채팅창 안**

8차시 Claude Code
AI가 **내 폴더에 직접 파일을 만들고** 명령을 실행한다. 결과가 웹 주소로 나온다

⇄ 달라지는 점은 하나

지금까지 AI는 **글**을 돌려줬다. 오늘 AI는 **손**이 있다. 파일을 만들고, 지우고, 명령을 친다. 그래서 오늘은 "무엇을 시킬지"와 함께 "**무엇을 승인할지**"를 배운다.

↻ 그대로 가져오는 것

1차시의 5요소 프롬프트, 2차시의 "지시문은 파일에 저장" 원칙(오늘은 `CLAUDE.md`), 그리고 준비 원칙(개인 계정 · 가상 데이터).

바이브 코딩이란

정의

코드를 직접 쓰지 않고, **원하는 것을 말로 설명**하고 AI가 만든 결과를 **보고**, 마음에 들 때까지 **다시 말하는** 방식. 2025년 초 Andrej Karpathy가 붙인 이름. 코드가 어떻게 생겼는지보다 **무엇이 나와야 하는지**에 집중한다.

전통적인 방법

문법을 배운다 → 코드를 쓴다 → 에러를 읽고 고친다 → 실행한다. 첫 홈페이지까지 보통 몇 주.

바이브 코딩

원하는 것을 말한다 → AI가 만든다 → 브라우저로 본다 → 고칠 점을 말한다. 첫 홈페이지까지 오늘 한 시간.

대신 필요한 것

문법 대신 **세 가지 습관**이 필요하다. 1 구체적으로 말하기 2 한 번에 하나씩 3 결과를 내 눈으로 확인하기. AI의 "다 됐습니다"는 확인이 아니다.

오늘의 결과물: 주소 하나



완성 기준 3개

✓ PC 브라우저에서 내 주소가 열린다
✓ 휴대폰에서도 열리고 깨지지 않는다
✓ 문구를 고치면 1~2분 안에 주소에 반영된다

왜 GitHub Pages인가

무료, 서버 없음, 공개 저장소에 `index.html`을 올리면 끝. 실습실 PC가 초기화돼도 **결과물은 GitHub에 남는다**.

소재: 나를 소개하는 한 페이지

닉네임 · 한 줄 소개 · 관심사 카드 3개 · 최근 활동 · 연락 링크. **가상 프로필**로 만든다(공개 주소이므로). 슬라이드 예시는 닉네임 "민트".

준비 원칙 (1 · 2차시와 동일)



개인 이메일

GitHub 가입 · Claude 로그인 모두 개인 계정



가상 프로필

닉네임 · 가상 관심사. 실명 · 소속 · 연락처 금지



전부 공개

저장소 · 홈페이지는 누구나 본다. 올리기 전 한 번 더 읽기



PC 초기화

실습실 PC는 수업 후 초기화. 결과는 GitHub에, 로그인은 수업 끝에 해제

Claude Code 계정

Claude Code는 **무료 플랜으로는 열리지 않는다**(Pro · Max · Team · Enterprise 또는 Console). 오늘은 강사의 **Console 조직에 초대된 계정**으로 로그인한다. 초대 메일을 미리 수락해 `platform.claude.com` 로그인이 되는지 확인.

5분 규칙

설치 · 로그인에서 5분 넘게 막히면 손을 든다. 에러 메시지는 **그대로 복사**해 둔다.



1부 설치

Git · gh · Claude Code

오늘 깔 것 3개



1. Git

파일의 **저장 지점**을 남기고 되돌린다. GitHub에 올릴 때도 이것을 쓴다



2. GitHub CLI (gh)

내 GitHub 계정과 연결. 저장소 만들기 · 올리기 · Pages 켜기를 **명령 한 줄**로



3. Claude Code

엔진. 위 두 도구를 **대신 쳐 주는** 쪽. 말로 시키면 git · gh 명령을 실행한다

순서와 이유

Git → gh → Claude Code. Git이 먼저 있어야 Claude Code가 Bash 도구를 쓸 수 있고, gh가 있어야 "GitHub에 올려줘"가 그 자리에서 된다. 미리 깔지 않으면 4부에서 멈춘다.

소요 시간

세 개 합쳐 5~10분. 전부 **명령 한 줄씩**이고 관리자 권한은 필요 없다. 설치 창이 뜨면 기본값으로 다음만 누른다.

터미널 열기 (Windows)



1. 열기

`Win + X` → "터미널" 또는 "Windows PowerShell". 시작 메뉴 검색도 된다



2. 확인

줄 앞이 `PS C:\Users\...>` 이면 PowerShell. `PS`가 없으면 CMD



3. 붙여넣기

명령을 복사한 뒤 `Ctrl + V` 또는 마우스 오른쪽 클릭, 그리고 `Enter`

⚠ 오늘 가장 흔한 실수: CMD에서 실행

오늘 명령은 모두 **PowerShell** 기준이다. CMD에서 실행하면 `'irm' is not recognized` 가 뜬다. 창을 닫고 PowerShell을 다시 연다. "Windows PowerShell (x86)"도 아니다. 이름에 `(x86)`가 없는 것을 연다.

터미널이란

마우스 대신 **글자로 명령**을 내리는 창. 오늘 직접 치는 명령은 설치 몇 줄과 `claude`뿐이다. 나머지는 Claude Code 안에서 한국어로 말한다.

설치 1 · 2: Git · gh — winget 두 줄

PowerShell에 한 줄씩 붙여넣고 Enter

```
winget install --id Git.Git -e --source winget
winget install --id GitHub.cli -e --source winget
```

winget이란

Windows 10 · 11에 내장된 **앱 설치 명령**. 처음 실행 시 약관 동의를 물으면 `Y`. 설치 창이 뜨면 기본값 그대로 진행.

winget을 못 찾으면

Microsoft Store에서 "앱 설치 관리자"를 업데이트하거나, 설치 파일을 직접 받는다: `git-scm.com/downloads/win`, `cli.github.com`. 둘 다 기본값으로 다음.

두 줄이 하는 일

`Git.Git` = Git for Windows(Git Bash 포함). Claude Code가 이것으로 명령을 실행한다. `GitHub.cli` = `gh` 명령. 로그인은 2부에서 한다.

설치 3: Claude Code — 한 줄, 그리고 새 창

PowerShell에 붙여넣기 (공식 설치 명령)

```
irm https://claude.ai/install.ps1 | iex
```

설치가 끝나면 터미널 창을 닫고 새로 연다

새 프로그램의 경로(PATH)는 **새 창**에서만 반영된다. "Claude Code successfully installed!"가 보였는데 `claude`를 못 찾으면 대부분 이 문제.

새 창에서 확인 세 줄

```
git --version      # git version 2.x.x
gh --version       # gh version 2.x.x
claude --version   # 2.1.xxx (Claude Code)
```

`winget install Anthropic.ClaudeCode`도 된다(수동 업데이트). 공식 설치 명령은 자동 업데이트. 오늘은 위 한 줄로 통일.

설치 4: 로그인 — Console 초대 계정으로

작업 폴더를 만들고 그 안에서 시작

```
mkdir $HOME\Desktop\mysite
cd $HOME\Desktop\mysite
claude
```

화면 순서

1) 로그인 방식 → **Anthropic Console account**
2) Sign in with your Console account → 브라우저에서 **초대 이메일**로 로그인 → 승인
3) 터미널에 `Login successful` → `Enter`
4) 폴더 신뢰 여부를 물으면 **예**

브라우저가 안 열리면

`c`를 누르면 로그인 주소가 복사된다. 브라우저에 붙인다. 로그인 후 **코드**가 보이면 터미널의 `Paste code here`에 붙인다.

Console 초대 계정이란

강사가 Console 조직에 초대한 이메일. 역할 「Claude Code」, 비용은 강사 조직 부담. 미수락이면 메일함(스팸함 포함)에서 먼저 수락. 계정 변경은 `/login`.

실습 1: 설치 · 로그인 체크리스트 (10분)

`prompts/01_install.txt` — 명령 전문과 문제 해결 명령

하나씩 눈으로 확인

✓ 터미널 줄 앞에 `PS`가 보인다 (PowerShell)
✓ `git --version` · `gh --version` · `claude --version` 세 줄이 모두 버전을 출력한다
✓ `mysite` 폴더 안에서 `claude`를 실행했다
✓ 브라우저 로그인이 끝나고 터미널에 `Login successful`이 보였다
✓ Claude Code 대화 화면이 열려 있다 (아래쪽 상태줄에 모델 · 폴더 이름)
✓ 한 줄 입력해 본다: "지금 폴더에 무엇이 있는지 알려줘" → 답이 온다

막히면 Claude에게

"`gh --version`이 안 돼. 이 에러가 났어: (그대로 붙여넣기). 원인을 알려줘."

5분 규칙

5분 넘게 막히면 손을 든다. 설치는 옆에서 화면을 한 번 보는 것이 가장 빠르다. 다음 슬라이드의 표를 먼저 본다.

안 될 때: 증상별 조치

화면에 뜬 말	원인	조치
`'irm' is not recognized`	CMD에서 실행함	창을 닫고 PowerShell을 연다
`'claude' is not recognized`	새 창을 안 열었음	터미널을 닫고 새로 연다. 그래도 안 되면 `prompts/01`의 PATH 명령
`Could not create SSL/TLS secure channel`	구형 Windows 10	`prompts/01`의 TLS 한 줄을 먼저 실행하고 재시도
`winget`을 못 찾음	앱 설치 관리자 없음	Microsoft Store에서 업데이트 또는 설치 파일 직접 다운로드
브라우저 로그인 후 터미널이 그대로	콜백 실패	브라우저의 코드를 터미널에 붙여넣기
로그인은 됐는데 "플랜에 포함되지 않음"	구독 계정 쪽으로 로그인	`/logout` 후 **Anthropic Console account**로 다시

공통 규칙: 에러는 그대로

빨간 글씨를 **한국어로 요약하지 말고** 그대로 복사해 둔다. 조교에게 보여 줄 때도, Claude에게 물을 때도 원문이 단서다. 자세한 규칙은 3부.

참고: 집에서 다시 설치할 때

Mac은 터미널에 두 줄

```
curl -fsSL https://claude.ai/install.sh | bash
brew install gh
```

Windows는 1부 명령 그대로. Mac의 Git은 터미널에 `git`을 치면 설치 안내가 뜬다. 순서는 `prompts/10_home_setup.txt`.

터미널이 부담스러우면: 데스크톱 앱

`claude.com/download`의 Claude 데스크톱 앱(Windows · Mac)에도 Claude Code가 들어 있다. claude.ai 구독(Pro 등)으로 로그인해 폴더를 열고 채팅처럼 쓴다. 오늘의 Console 계정은 터미널 `claude`에서 쓴다. 명령과 습관은 그대로 통한다.

실습실과 다른 점

내 PC에서는 설치와 로그인이 **한 번**이면 된다. 다음부터는 폴더에서 `claude`만 친다. 오늘 만든 저장소는 `gh repo clone 아이디/아이디.github.io`로 받아 이어서 고친다(정리 슬라이드).



2부 Claude Code 기초

명령어 몇 개와 GitHub 계정

Claude Code란

한 줄 정의

터미널에서 실행되는 Claude. **내 폴더의 파일을 읽고, 만들고, 고치고, 명령을 실행**한다. 모델과 계정은 claude.ai와 같다. 다른 점은 "손"이 있다는 것.

claude.ai 채팅 (1 · 2차시)

- 답을 **글**로 준다
- 파일이 필요하면 내가 복사해 저장한다
- 내 컴퓨터를 건드리지 않는다

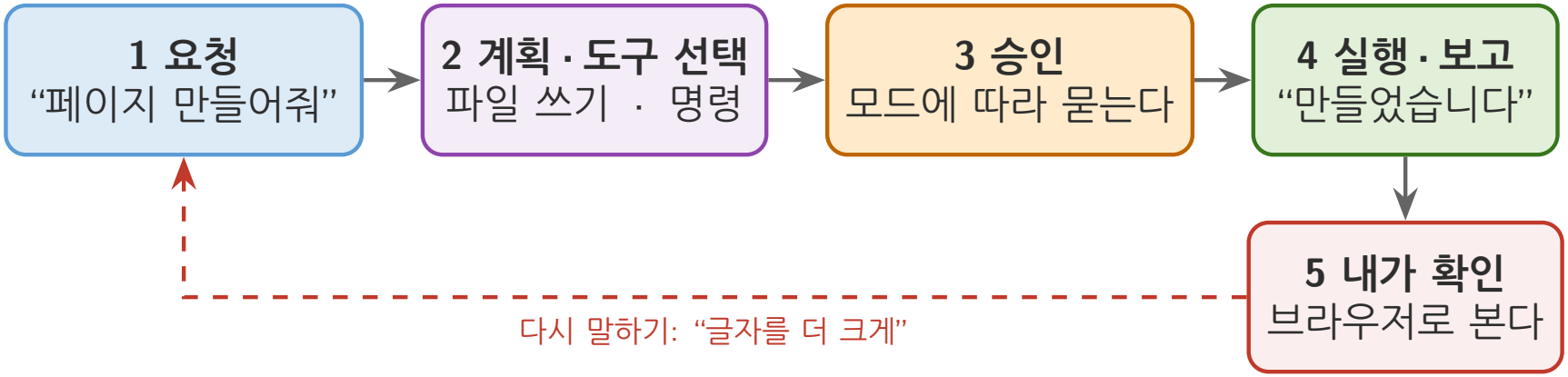
Claude Code (오늘)

- **파일을 직접** 만들고 고친다
- **명령을 직접** 실행한다 (git, gh, 브라우저 열기)
- 결과를 확인하고 스스로 다시 고친다

그래서 필요한 장치

손이 있으니 **어디까지 맡길지**를 정해야 한다. Claude Code는 파일을 바꾸거나 명령을 실행하기 전에 **권한 모드**에 따라 묻거나, 검토용 모델이 대신 검토한다. 다음 두 장.

요청 하나가 처리되는 흐름



"도구"란

파일 읽기 · 쓰기 · 편집, 명령 실행(Git Bash · PowerShell), 웹 페이지 읽기 · 검색. Claude가 상황에 맞게 골라 쓴다. 화면에 어떤 도구를 쓰는지 표시된다.

빨간 칸은 사람이 한다

5번을 건너뛰지 않는다. "만들었습니다"는 보고이지 확인이 아니다. 브라우저를 열어 내 눈으로 본 뒤 다음 말을 한다.

권한 모드: 누가 승인하나

모드	상태줄 표시	뜻
자동 (auto)	`auto mode on`	**기본값.** 검토용 모델이 대신 검토. 대부분 묻지 않고 진행, 위험한 것(삭제 · 외부 전송)만 묻는다
수동 (manual)	`manual mode on`	파일 수정 · 명령 실행 전 **매번 나에게** 묻는다
편집 자동 수락	`accept edits on`	파일 수정은 묻지 않음. 명령 실행은 묻는다
계획 (plan)	`plan mode on`	**읽기만.** 무엇을 할지 계획을 먼저 보여 주고 승인을 기다린다

⌨ 바꾸는 법: `Shift+Tab`

누를 때마다 순환한다: 자동 → 수동 → 편집 자동 수락 → 계획 → 자동. 화면 아래 상태줄에 현재 모드가 보인다. 큰 작업 전에는 계획(plan)으로 한 번 보고 시작하면 방향 착오를 줄인다.

✔ 오늘의 권장

실습 2 · 3은 **수동**으로 놓고 승인 창을 직접 읽어 본다(어떤 일이 벌어지는지 배우는 단계). 4부 홈페이지 만들기부터는 **자동**으로 돌려 속도를 낸다. 자동이어도 push · 삭제 같은 것은 묻는다.

승인 창 읽는 법

이런 창이 뜬다 (수동 모드)

`Write index.html` — "이 파일을 만들까요?" 선택지: **Yes** / **Yes, and don't ask again for ...** / **No**. 화살표로 고르고 `Enter`. 명령 실행이면 실행할 명령이 그대로 보인다.

Yes
이번 한 번 허용. 기본 선택

Yes, and don't ask again
같은 종류는 앞으로 묻지 않음. 권한 규칙이 된다(`/permissions`에서 확인)

No
멈춘다. 이어서 "대신 이렇게 해줘"라고 말하면 된다

읽을 것 세 가지

1. 어떤 **도구**인가(파일 쓰기? 명령 실행?)
2. **무엇에** 하는가(어느 파일, 어떤 명령)
3. **되돌릴 수 있나**(삭제 · push · 외부 전송은 천천히). 모르면 No를 누르고 "이 명령이 뭐 하는 거야?"라고 묻는다.

실습 2: 첫 대화 (5분)

prompts/02_first_chat.txt ⌨ 준비: mysite에서 claude 실행 중, Shift+Tab으로 상태줄을 manual mode on으로.

1) 첫 메시지 (그대로 입력)

안녕! 이 폴더에 hello.txt 파일을 만들고
그 안에 "내 첫 바이브 코딩" 이라고 써줘.

2) 확인 · 3) 되돌리기

방금 만든 파일 내용을 보여줘.
방금 만든 hello.txt를 지워줘.

완료 기준

✓ 승인 창을 **읽고** Yes를 눌렀다 ✓ 파일 탐색기에서 hello.txt가 보였다 ✓ 말로 지웠고, 지울 때도 승인 창이 떴다

슬래시 명령: 자주 쓰는 것만

명령	언제 쓰나
`/help`	명령 목록과 도움말. 막히면 먼저
`/clear`	대화를 비우고 새 작업 시작. **작업이 바뀌면**
`/init`	이 폴더용 `CLAUDE.md`(규칙 파일) 초안 만들기 (3부)
`/model`	모델 바꾸기. 오늘은 기본값 그대로
`/permissions`	"다시 묻지 않음"으로 쌓인 권한 규칙 보기 · 지우기
`/usage`	이번 세션의 토큰 · 비용 확인
`/resume`	이전 대화 이어하기 (터미널을 닫았다 열었을 때)
`/rewind`	코드 · 대화를 이전 시점으로 되감기
`/exit`	종료 (`Ctrl+D` 두 번도 같음)

외울 필요 없다

입력창에 /만 치면 목록이 뜬다. `Tab`으로 자동 완성. 버전에 따라 명령이 더 있으니 `/help`가 최신 목록.

슬래시 명령: 보기 · 묻기 · 조절하기

명령	하는 일	알아 둘 점
`/context`	지금 대화가 컨텍스트(상자)를 얼마나 채웠는지 색 격자로 보여 준다	무엇이 자리를 차지하는지도 나온다. 많이 찼으면 `/compact`나 `/clear` (3부)
`/btw` 질문	곁질문. 답은 겹쳐 뜨는 창에 나오고 **대화 기록에 남지 않는다**	Claude가 일하는 중에도 된다. 파일 읽기 · 명령 실행은 못 하고 지금까지의 대화로만 답한다
`/effort`	사고 수준 고르기. `low` · `medium` · `high` · `xhigh` · `max`	높을수록 깊게 생각하고, 느리고, 비용이 더 든다. 정한 값은 다음 실행에도 유지(`max`는 이번 세션만)

써 보기

```
/context
/btw 방금 만든 파일 이름은?
/effort    슬라이더가 열린다
/effort high    바로 지정
```

오늘은

사고 수준은 기본값 그대로 둔다. 홈페이지 한 장에는 충분하다. `/context`와 `/btw`는 실습 3이 끝날 때 한 번씩 쳐 본다.

멈추기 · 되돌리기

키	하는 일
`Esc`	진행 중인 작업을 **멈춘다**. 원치 않는 방향이면 즉시
`Esc` 두 번	**되감기** 메뉴. 이전 메시지 시점으로 코드 · 대화를 복구 (`/rewind`와 같음)
`Ctrl+C`	입력 중인 글 지우기, 또는 중단
↑	이전에 입력한 문장 불러오기
`Shift+Tab`	권한 모드 바꾸기

말로도 된다

"방금 변경 되돌려줘" "`index.html`을 10분 전 상태로 돌려줘". Claude Code는 파일을 바꿀 때마다 **체크포인트**를 남겨 두므로 돌아갈 수 있다.

가장 중요한 하나

`Esc` = "잠깐 멈춰". 이상한 파일을 만들기 시작하면 다 끝나기를 기다리지 말고 멈춘 뒤 다시 말한다.

실습 3: 맛보기 페이지 (7분)

prompts/03_taste_page.txt

1) 만들고 열기

이 폴더에 `index.html` 하나를 만들어줘.
화면 가운데에 "안녕하세요, 첫 페이지입니다"를 크게 쓰고,
배경은 연한 파란색으로. 만든 뒤 기본 브라우저로 열어줘.

2) 한 줄 고치고 새로고침(F5)

글자를 진한 파랑으로 바꾸고, 그 아래에 오늘 날짜를 작게 넣어줘.

이것이 오늘의 리듬

말하기 → 보기 → 다시 말하기. 한 번에 하나만 바꾸고 눈으로 확인한다. 4부에서는 이 리듬으로 진짜 홈페이지를 만든다.

브라우저가 안 열리면

파일 탐색기에서 `Desktop` → `mysite` → `index.html`을 더블클릭. 이후 고칠 때는 브라우저에서 F5만 누르면 된다.

끝나면 한 번씩: `/context` (상자가 얼마나 찼나) `/btw` 방금 만든 파일 이름은? (곁질문)

GitHub란

한 줄 정의

코드와 파일을 **보관**하고 **이력**을 남기는 인터넷 서비스. 무료로 **정적 홈페이지를 공개**하는 기능(GitHub Pages)이 붙어 있다. 무료 플랜의 Pages는 **공개 저장소만** 된다(내용 · 이력 모두 공개).



저장소 (repository)
프로젝트 폴더 하나. 오늘은 아이디.github.io 한 개



커밋 (commit)
저장 지점. 사진 찍기. "무엇을 왜 바꿨는지" 한 줄이 붙는다



push
내 PC의 커밋을 GitHub로 올리기. 올라간 뒤 Pages가 주소에 반영

오늘 쓰는 이유 둘

1 실습실 PC가 초기화돼도 **GitHub에 남는다** 2 **Pages**로 `https://아이디.github.io` 주소가 생긴다. 명령은 외우지 않는다. Claude에게 "올려줘"라고 말한다.

GitHub 가입: 아이디가 곧 주소



1. 접속

`github.com/signup`



2. 이메일

개인 이메일 + 비밀번호
(다른 곳과 다르게)



3. 아이디

username.
소문자 · 숫자 · 하이픈.
이것이 주소가 된다



4. 인증

퍼즐 확인 → 이메일로
온 코드 입력 → 완료

아이디 정하기 전에

내 홈페이지 주소는 `https://`**아이디**`.github.io`. 예: 아이디 `mint-note` → `https://mint-note.github.io`. 실명 · 소속 · 군번 같은 개인정보를 넣지 않는다. 나중에 바꾸면 주소도 바뀐다.

참고

인증 메일이 안 오면 스팸함. "Continue with Google"로 Google 계정 가입도 된다. 이미 계정이 있으면 로그인만 하고 아이디를 메모해 둔다.

gh 로그인과 git 이름표

새 터미널 창을 하나 더 열고 (Claude Code 창은 그대로)

```
gh auth login
```

질문에 이렇게 답한다 (화살표 · Enter)

GitHub.com → **HTTPS** → Authenticate Git? **Y** → **Login with a web browser** → **8자리 코드** 복사 후 Enter → 브라우저에 코드 입력 → **Authorize**. 확인: `gh auth status`에 `Logged in`.

이름표 두 줄 (커밋에 "누가"로 남는다)

```
git config --global user.name "닉네임"
git config --global user.email "가입한이메일@example.com"
```

⚠ `gh auth login`은 브라우저 인증이 필요한 **대화형** 명령이라 직접 친다. 이름표는 Claude에게 "git 이름표를 닉네임 민트로 설정해줘"라고 시켜도 된다.

실습 4: GitHub 가입 · gh 로그인 (8분)

prompts/04_github_gh.txt

체크리스트

✓ github.com/signup 가입 완료, 이메일 인증까지
✓ 내 아이디를 메모했다 → 내 주소는 https://아이디.github.io
✓ 새 터미널에서 gh auth login 완료, gh auth status에 Logged in
✓ git config 이름표 두 줄 입력
✓ Claude Code 창으로 돌아와 확인: "gh auth status 결과를 보여줘"

공용 PC 주의

브라우저의 "비밀번호 저장" 제안은 **거절**. 수업 끝에 github.com 로그아웃, gh auth logout, Claude Code /logout까지(정리 슬라이드 체크리스트).

막히면

코드 입력 페이지가 안 뜨면 github.com/login/device를 직접 열어 코드를 넣는다. 인증 메일은 스팸함 확인.

명령

`claude` — 폴더에서 시작
`/help` — 도움말
`/clear` — 새 작업 시작
`/init` — `CLAUDE.md` 초안
`/permissions` — 권한 규칙
`/usage` — 사용량
`/resume` — 이어하기
`/rewind` — 되감기
`/context` — 상자 사용량
`/btw` — 곁질문 (기록에 안 남음)
`/effort` — 사고 수준
`/exit` — 종료

키 · 습관

`Esc` — 멈추기
`Esc Esc` — 되감기
`Shift+Tab` — 권한 모드
↑ — 이전 입력
승인 창은 **읽고** 누른다
한 번에 **하나**만 바꾼다
바꾼 뒤 **브라우저로 확인**
에러는 **그대로** 붙여넣기
"되돌려줘"로 복구

딱 둘만 기억한다면

막히면 `/help`, 멈추려면 `Esc`. GitHub 쪽은 "올려줘"라고 말하면 Claude가 git · gh 명령을 친다. 사람은 **아이디**를 정확히 주고 **승인 창**을 읽는다.



3부 팁

막히지 않게, 막혔을 때

잘 되는 요청 vs 안 되는 요청

✗ 안 되는 요청

"홈페이지 만들어줘"
"예쁘게 해줘"
"다 고쳐줘"
"안 돼. 왜 안 돼?"

✓ 되는 요청

"나를 소개하는 1페이지. 닉네임 · 소개 · 관심사 3개 · 연락 링크. 파일 하나로."
"제목을 더 크게, 카드 사이 간격을 넓게"
"관심사 카드만 동그랗게. 다른 곳은 그대로"
"이 에러가 났어: (원문). 원인부터 알려줘"

1차시 5요소가 그대로 통한다

과제(무엇을) + **형식**(모양 · 구성) + **제약**(하지 말 것 · 조건)이 핵심. 역할은 생략해도 된다. 맥락은 `CLAUDE.md`에 넣어 두면 매번 안 써도 된다.

한 번에 하나. 세 가지를 한 문장에 넣으면 무엇이 잘못됐는지 못 찾는다. 바꾸고 → 새로고침 → 확인 → 다음.

막혔을 때 4가지

1. 에러 붙여넣기

빨간 메시지를 **그대로 복사**해 붙이고 "원인이 뭐야?". 요약하지 않는다.

2. 되돌리기

"방금 변경 되돌려줘" 또는 `Esc Esc`. 망가진 상태에서 계속 고치지 않는다.

3. 작게 쪼개기

한 번에 안 되면 단계를 나눈다. "먼저 상단만", "다음에 카드만".

4. 다르게 말하기

같은 말을 반복하지 않는다. 더 구체적으로, 예시를 들어, 또는 "내가 원하는 건 ○○인데 지금은 △△야".

기억할 것

막히는 것은 정상이다. 넷 중 하나로 대부분 풀린다. 그래도 안 되면 `/clear`로 새 대화를 열고 처음부터 **현재 상태**를 설명한다.

에러는 그대로, 상황은 다섯 칸으로

한국어로 요약하지 않는다

"뭔가 빨간 게 떴어"에는 답이 없다. 에러 원문에 있는 **파일 이름 · 줄 번호 · 오류 이름**이 단서다. 길면 가운데를 ...(중략)...으로 자르되 앞뒤는 그대로.

칸	적을 것
증상	무엇이 잘못 보이나. 한 줄
재현	무엇을 했을 때 (어떤 요청 뒤에, 어떤 버튼을 눌렀을 때)
기대	원래 어떻게 돼야 하나
실제	화면에 나온 것. **원문 그대로**
해 본 것	이미 시도한 것 (그래야 같은 것을 다시 안 한다)

고치기 전에 원인부터 받는 문장

"고치지 말고, 원인 가설을 두세 개 말하고 각각 어떻게 확인할지 먼저 알려줘." → 그럴듯한 추측 수정으로 멀쩡한 곳까지 건드리는 것을 막는다. 문장은 `prompts/08_stuck.txt`.

대화가 길어지면 나빠진다

이유

AI는 답할 때마다 **지금까지의 대화 전체**를 다시 읽는다. 길어지면 앞에서 정한 규칙을 잊고, 옛 요구와 새 요구를 섞고, 느려진다. 2차시의 "컨텍스트는 상자"와 같은 이야기.

선택	이럴 때
그냥 계속	작은 수정만 남았고 흐름이 선명하다
`/context`	얼마나 찼는지 모르겠다 (색 격자로 확인)
`/btw`	곁질문이 있다 (기록에 남지 않는다)
`/compact`	**같은 작업**인데 대화만 길어졌다 (요약해서 이어감)
`/clear`	**작업이 바뀌었다.** 새 대화에서 현재 상태부터 다시 설명
파일로 빼기	매번 다시 설명하는 규칙이 있다 → `CLAUDE.md` (다음 장)

오늘 적용

실습 3(맛보기)이 끝나면 `/clear`. 4부 홈페이지는 새 대화에서 시작한다. 배포까지 끝난 뒤 수정 작업도 필요하면 `/clear` 후 "`index.html`의 ○○를 바꿔줘".

CLAUDE.md: 매번 말하지 않기

무엇인가

폴더 안의 CLAUDE.md에 적어 둔 규칙은 **새 대화마다 자동으로 읽힌다**. 2차시 스킬이 "지시문을 파일로"였다면, 이것은 "이 폴더의 규칙을 파일로".

오늘 폴더용 예시 (5줄이면 충분)

```
# 이 폴더의 규칙
- 답변은 한국어로 한다.
- 파일은 index.html 하나로 유지한다. 외부 라이브러리는 최소로.
- 파일을 바꾼 뒤에는 기본 브라우저로 열어 보여준다.
- 실명, 전화번호, 주소 같은 개인정보는 넣지 않는다.
```

만드는 법 둘

/init (Claude가 폴더를 보고 초안) 또는 "prompts/09의 내용으로 CLAUDE.md 만들어줘".

분량

짧을수록 지켜진다. 같은 지적을 **두 번** 했을 때 한 줄 추가. 비밀번호 · 키는 절대 넣지 않는다.

intent.md: 만들기 전에 적는다

무엇인가

만들기 전에 **무엇을 · 왜 · 어떤 제약으로** 원하는지 적어 두는 파일. Anthropic「AI-native SDLC 플레이북」(2026.8)에서 개발의 첫 산출물로 쓰는 이름이다.

다섯 칸	적을 것
문제	지금 무엇이 불편한가. 왜 만드나
제안된 결과	다 만들면 무엇이 보이나
영향을 받는 사용자와 시스템	누가 보나. 어디에 올리나
제약 사항	하지 않을 것. 지킬 조건
미해결 질문	아직 못 정한 것

CLAUDE.md와 다른 점

`CLAUDE.md`는 폴더의 규칙이고 **자동으로** 읽힌다. `intent.md`는 이번 작업의 의도이고 자동으로 읽히지 않는다. Claude Code 기능이 아닌 약속이라 "`intent.md`를 읽고"라고 말한다.

💬 **쓰는 법**: 직접 타이핑하지 않는다. "`intent.md`를 쓸 거야. 다섯 칸을 하나씩 나에게 물어보고 파일로 저장해줘." 저장된 파일은 읽고 고친다.

intent.md 예시와 계획 모드

오늘 홈페이지용 예시

```
# Intent: 민트의 소개 페이지
작성: 민트. 상태: 초안.
## 문제
나를 소개할 주소가 없어서 매번 말로 설명한다.
## 제안된 결과
주소 하나를 열면 소개, 관심사 3개, 연락 링크가 보인다.
## 영향을 받는 사용자와 시스템
처음 만난 사람이 휴대폰으로 본다. GitHub Pages에 올린다.
## 제약 사항
index.html 하나. 실명, 소속, 연락처, 사진은 넣지 않는다.
## 미해결 질문
배경은 밝게 할까 어둡게 할까? 사진 자리에 무엇을 넣을까?
```

계획 모드에 건네기

1) Shift+Tab으로 상태줄을 plan mode on으로 바꿔. 바뀌는 파일, 작업 순서, 확인 방법을 적어줘.” **2)** “intent.md를 읽고 구현 계획을 보여줘.” **3)** 계획을 읽고 고칠 곳을 말한다. 승인하면 만들기 시작

prompts/12_intent_md.txt 플레이북의 흐름은 intent.md → spec.md → 계획 모드 → plan.md → 코드. 오늘은 한 장짜리라 spec.md · plan.md는 건너뛴다. 실습 5-1을 이 순서로 시작해도 된다(선택).

공개 저장소 보안: 넣지 않는 것

전제

공개 저장소와 GitHub Pages는 **전 세계 누구나** 본다. 한 번 올라간 파일은 지워도 **수정 이력에 남는다**. 올리기 전에 한 번 더 읽는 습관.

넣지 않는다

✘ 실명 · 소속 · 직책 · 군번
✘ 전화 · 주소 · 개인 이메일
✘ 내 사진 · 타인 사진(초상권)
✘ 비밀번호 · API 키 · 토큰
✘ 실제 업무 자료 · 내부 문서

대신

✓ 닉네임 · 가상 프로필
✓ 연락은 GitHub 프로필 링크
✓ 무료 이미지(Unsplash 등)
✓ 키가 필요한 기능은 오늘 안 만든다
✓ 예시 데이터는 지어낸다

근거 (1차시와 동일)

국가정보원 · 국가사이버안보센터 「챗GPT 등 생성형 AI 활용 보안 가이드라인」(2023.6). 실제 업무 자료는 소속 조직의 보안 지침이 우선한다. Claude Code에게 "비밀 파일은 읽지 마"라고 규칙을 줄 수도 있다(`/permissions`).

사용량 관리

Console 조직은 쓴 만큼 낸다

오늘의 비용은 강사 Console 조직의 크레딧에서 **토큰 수만큼** 차감된다. 사용량은 **대화 길이 · 파일 크기 · 모델 · 사고 수준**에 비례. `/usage`로 이번 세션의 토큰과 비용을 본다.

절약 3가지

1. 요청은 작게, 한 번에 하나
2. 작업이 바뀌면 `/clear` (긴 대화는 매 번 다시 읽힌다)
3. 큰 이미지·긴 파일을 대화에 붙이지 않는다

지출 한도에 걸리면

강사가 워크스페이스에 정한 한도에 닿으면 모두의 응답이 멈춘다. 손을 들어 알린다. 만든 파일은 그대로 남는다.

모델 · 사고 수준: 오늘은 둘 다 기본값 그대로. 홈페이지 한 장에는 충분하다. `/model`로 모델을, `/effort`로 사고 수준을 바꾼다. 큰 모델, 높은 수준일수록 비용이 빨리 는다.



4부 홈페이지 만들기

만들고, 올리고, 고친다

오늘 만드는 페이지

상단 닉네임 · 한 줄 소개 · (배경 이미지)

소개 3~4문장

관심사 카드 3개 [사진] [독서] [달리기]

최근 활동 3줄 (날짜 · 한 줄)

연락 GitHub 프로필 링크

하단 · 만든 날 · 사용한 이미지 출처

규칙

- 파일은 `index.html` **하나** (이미지는 인터넷 주소로)
- 휴대폰 화면에서도 깨지지 않게
- 한국어, 닉네임 · 가상 프로필
- 밝고 깔끔한 톤, 색은 한 계열

왜 파일 하나인가

더블클릭으로 열리고, 올릴 것도 하나, 깨질 곳도 하나. 익숙해지면 파일을 나눈다(사례 슬라이드).

시간 배분 (45분)

레퍼런스 · 프롬프트 10 → 만들기 · 개선 15 → 올리기 · Pages 12 → 재배포 · 사례 8

무료 레퍼런스 사이트 1: 템플릿 · 영감

사이트	무엇이 있나	조건
HTML5 UP `html5up.net`	반응형 HTML 템플릿 40여 종. 완성도 높음	무료, CC BY 3.0 (하단 출처 표기 유지)
TemplateMo `templatemo.com`	HTML 템플릿 600여 종, 로그인 없이 다운로드	무료, 개인 · 상업 사용 가능
Start Bootstrap `startbootstrap.com`	Bootstrap 기반 템플릿 · 컴포넌트	무료 다수 (MIT), 일부 유료
Free CSS `free-css.com`	여러 출처의 템플릿 모음	템플릿마다 라이선스 확인
One Page Love `onepagelove.com`	한 페이지 사이트 사례 9천여 개	보기 무료. 구성 · 분위기 참고
Land-book `land-book.com`	랜딩 페이지 갤러리	보기 무료. 영감용

쓰는 법: 링크를 주고 "이 느낌으로"

템플릿 파일을 그대로 쓰는 것보다 **"이 페이지의 구성과 분위기로 새로 만들어줘"**가 쉽다. Claude Code는 주소를 읽을 수 있다. 파일을 그대로 쓸 때는 라이선스(출처 표기)를 지킨다. 목록과 문장은 `prompts/11_reference_sites.txt`.

무료 레퍼런스 사이트 2: 이미지 · 글꼴 · 아이콘 · 색

사이트	무엇이 있나	조건
Unsplash `unsplash.com`	고화질 사진. 주소만 복사해 배경으로	무료, 상업 사용 가능, 출처 표기 권장
Pexels `pexels.com`	사진 · 짧은 영상	무료
Google Fonts `fonts.google.com`	웹 글꼴 (Noto Sans KR 등)	무료, 링크 한 줄로 사용
Pretendard	한국어 화면용 글꼴. 가독성 좋음	무료 (OFL), CDN 한 줄
Font Awesome `fontawesome.com`	아이콘 2천여 개	무료 버전 있음
Coolors `coolors.co`	색 조합 생성기. 5색 한 세트	무료

Claude에게 이렇게

"상단 배경은 Unsplash의 노트 · 책상 느낌 무료 사진 주소를 골라 넣어줘. 글꼴은 Pretendard, 카드 아이콘은 Font Awesome."

사진 주의

내 사진 · 타인 사진 · 검색해서 찾은 사진은 넣지 않는다. 위 사이트의 무료 사진만.

프롬프트 설계: 5요소를 홈페이지에

요소	홈페이지에서는	예
역할	(생략 가능)	"웹 디자이너처럼" 정도. 없어도 된다
맥락	누구의 페이지, 목적	닉네임 "민트"의 개인 소개. 취미(사진 · 독서 · 달리기) 중심
과제	만들 것	한 페이지 소개 사이트, `index.html` 하나
형식	구성 · 느낌	상단 / 소개 / 관심사 카드 3 / 최근 활동 / 연락. 밝은 파란 계열
제약	조건 · 하지 말 것	파일 하나, 휴대폰 대응, 외부 라이브러리 최소, 개인정보 없음

첫 프롬프트는 길게 한 번, 그 뒤는 한 줄씩

처음에 구성 · 느낌 · 제약을 다 적으면 골격이 한 번에 나온다. 그 다음부터는 **한 번에 하나씩** 고친다. 처음부터 완벽할 필요 없다. 고치는 것이 오늘의 연습이다.

다음 장 프롬프트에서 **닉네임** · **한 줄 소개** · **관심사 3개** · **색**만 바꾼다. 나머지는 그대로 붙여 넣어도 된다.

실습 5-1: 만들기 (7분)

prompts/05_homepage_build.txt 시작 전 /clear. 권한 모드는 Shift+Tab으로 **자동(auto)**.

프롬프트 (닉네임 · 소개 · 관심사 · 색만 내 것으로)

나를 소개하는 한 페이지 홈페이지를 만들어줘.
- 파일은 index.html 하나. 설치 없이 더블클릭으로 열리게.
- 닉네임 "민트", 한 줄 소개 "기록하고 정리하는 사람".
- 구성: 상단 인사 영역 / 소개 / 관심사 카드 3개(사진, 독서, 달리기) / 최근 활동 3줄 / 연락(GitHub 프로필 링크).
- 밝고 깔끔하게, 파란색 계열. 휴대폰 화면에서도 깨지지 않게.
- 글꼴은 Pretendard. 이미지는 Unsplash 무료 사진 주소만.
- 실명, 전화, 주소 같은 개인정보는 넣지 마.

기존 index.html은 덮어써도 돼. 만든 뒤 기본 브라우저로 열어줘.

관찰

자동 모드에서는 승인 창이 거의 뜨지 않고 파일이 만들어진다. 브라우저가 열리면 **위에서 아래로 한 번 다 읽는다**. 틀린 문구 · 빠진 구성이 있으면 다음 장에서 한 줄씩 고친다.

실습 5-2: 반복 개선 (8분)

한 줄씩. 매번 F5로 확인 (최소 3번)

상단 배경을 연한 그라데이션으로 바꿔줘.
관심사 카드마다 어울리는 아이콘을 하나씩 넣어줘.
휴대폰 폭에서는 카드가 세로로 쌓이게 해줘.
"최근 활동"에 날짜를 붙이고 최신순으로 정렬해줘.
하단에 사용한 사진 출처(Unsplash)를 작게 적어줘.

레퍼런스를 쓰는 문장

`https://html5up.net/` 의 "Story" 템플릿처럼 상단을 화면 가득 큼직하게, 아래 카드는 지금대로 유지해줘.

마음에 안 들면

"방금 변경 되돌려줘". 되돌린 뒤 **다르게** 말한다. 같은 문장을 반복하지 않는다.

휴대폰 화면 확인

브라우저 창을 좁게 줄여 보거나, "휴대폰 폭으로 스크린샷을 찍어 보여줘"라고 시킨다.

실습 5 완료 기준과 흔한 상황

상황	이렇게 말한다
결과가 마음에 안 든다	“방금 변경 되돌려줘”
무엇이 바뀌었는지 모르겠다	“방금 무엇을 바꿨는지 세 줄로 요약해줘”
시키지 않은 곳까지 바꿨다	“다음부터는 내가 말한 부분만 고쳐줘”
이미지가 안 뜬다	“이미지 주소가 열리는지 확인하고 안 열리면 다른 걸로 바꿔줘”
글자가 깨진다(한글)	“문자 인코딩을 UTF-8로 지정해줘”
화면이 안 열린다	파일 탐색기에서 `index.html` 더블클릭

완료 기준

✓ 브라우저에서 열린다 ✓ 닉네임 · 소개 · 관심사 3개 · 활동 · 연락이 다 있다 ✓ 창을 좁혀도 깨지지 않는다 ✓ 말로 **3번 이상** 고쳤다 ✓ 개인정보가 없다

시간이 남으면

다크 모드 토글, 방문자에게 인사하는 작은 애니메이션, 관심사 카드 클릭 시 설명 펼치기. 어느 것이든 **한 줄**로 시키고 확인한다.

GitHub에 올리기: 무엇이 일어나나



명령은 Claude가 친다. 사람이 하는 일은 둘

1 저장소 이름을 **정확히** 준다: 아이디.github.io (소문자) 2 push · 저장소 생성 승인 창을 **읽고** 누른다. 밖으로 나가는 작업은 자동 모드에서도 묻는 경우가 있다.

저장소 이름이 곧 주소

저장소 아이디.github.io
https://아이디.github.io
저장소 mysite
https://아이디.github.io/mysite/

사용자 사이트

이름이 아이디.github.io인 저장소는 GitHub이 "사용자 사이트"로 취급한다. 계정당 하나. 오늘은 이것을 쓴다.

실습 6-1: 올리기 (6분)

prompts/06_deploy.txt

프롬프트 ("아이디"를 내 GitHub 아이디로)

이 폴더를 git 저장소로 만들고 첫 커밋을 해줘.
그다음 gh로 내 GitHub에 "아이디.github.io" 라는 이름의
공개(public) 저장소를 만들고 push 해줘.
아이디는 gh auth status로 확인해서 정확히 써.
끝나면 저장소 주소와 홈페이지 주소를 알려줘.

뒤에서 도는 명령 (참고용. 외우지 않는다)

```
git init  &&  git add .  &&  git commit -m "첫 페이지"
gh repo create 아이디.github.io --public --source=. --push
```

승인 창

gh repo create · git push는 **밖으로 나가는** 작업. 저장소 이름과 --public이 맞는지 읽고 Yes.

확인

브라우저에서
github.com/아이디/아이디.github.io
열기. index.html이 보이면 성공.

실습 6-2: Pages 켜기 (5분)

먼저 주소로 접속해 본다

`https://아이디.github.io`. 사용자 사이트 저장소는 Pages가 **자동으로 켜지는 경우가 많다**. 1~2분 뒤 열리면 끝. 아니면 A 또는 B.

A. 웹에서 (눈으로 확인)

저장소 페이지 → **Settings** → 왼쪽 **Pages** → Build and deployment의 Source: **Deploy from a branch** → Branch: **main**, 폴더 **/ (root)** → **Save**.

B. 말로

"이 저장소의 GitHub Pages를 main 브랜치 루트로 켜줘." → Claude가 `gh api`로 설정한다. 명령은 `prompts/06`에 있다.

반영 시간

보통 1~2분, 길면 10분. Settings → Pages 상단에 "Your site is live at ..."가 뜨면 완료. 안 뜨면 강력 새로고침(`Ctrl+Shift+R`), **Actions** 탭에서 초록 체크 확인.

404가 뜨면 셋 중 하나: 저장소 이름이 `아이디.github.io`인가(소문자) · `index.html`이 저장소 **맨 위**에 있는가 · 저장소가 **Public**인가.

확인과 공유



PC

`https://아이디.github.io` 접속. 로컬 파일과 같게 보이는가



휴대폰

같은 주소를 휴대폰에서. 카드가 세로로 쌓이고 글자가 잘리지 않는가



옆사람

주소를 서로 열어 본다. 남의 화면에서 열리면 진짜 공개된 것

여기까지 되면

오늘의 목표는 끝났다. 남은 것은 **고치고 다시 올리는 루프**(다음 장). 이 루프가 익숙해지면 무엇이든 같은 방식으로 만들고 배포할 수 있다.

공개 전 마지막 점검

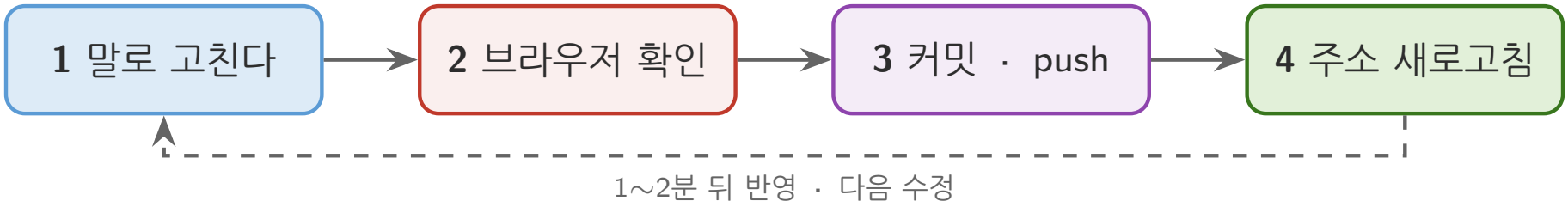
페이지를 처음부터 끝까지 읽으며 **실명 · 소속 · 연락처 · 사진**이 없는지 본다. 있으면 지우고 다시 올린다(이력에는 남으므로 처음부터 넣지 않는 것이 원칙).

실습 7: 고치고 다시 올리기 (5분)

prompts/07_update_loop.txt

프롬프트

한 줄 소개를 "정리하는 습관을 만드는 사람"으로 바꿔줘.
브라우저로 확인한 뒤, 무엇을 왜 바꿨는지 한 줄 메시지로
커밋하고 push 해줘.



이력이 남는다

커밋마다 "무엇을 왜"가 기록된다. 잘못 올렸으면 "이전 커밋 상태로 되돌리고 push 해줘".

빨간 칸(2)은 사람

확인 없이 push하면 틀린 문구가 공개된다.
로컬에서 본 뒤 올린다.

사례: 실제로 이렇게 만든 홈페이지

AutoML Lab 홈페이지 (경기대 연구실)

`minercode625.github.io/automl-lab-homepage`. Claude Code로 만들고 GitHub Pages로 배포. 오늘과 같은 순서(말로 시키고, 확인하고, 커밋). 다른 점은 **규모**와 **파일 구조**.

만든 순서

1) 스펙: "무엇을 · 왜 · 누가 고치나"를 문서로 먼저
2) 계획: 할 일을 작은 단계로 나눈 체크리스트
3) 구현: 단계마다 만들고 브라우저로 확인
4) 문서: 수정 안내서 · 배포 안내서도 Claude가 작성

구조: 뼈대와 내용 분리

`index.html` · CSS · JS는 뼈대. 멤버 · 논문 · 소식은 `data/*.yml` 텍스트 파일. 연구실 학생은 **GitHub 웹에서 yml 한 줄만 고치면** 1분 안에 반영된다. 코드는 몰라도 된다.

오늘 페이지의 다음 단계가 이것. 내용이 늘면 "내용을 별도 파일로 빼고, 파일만 고치면 반영되게 해줘". 여럿이 고치면 "수정 안내서(`UPDATE_GUIDE.md`)도 써줘".

더 해볼 것

페이지 나누기
소개 · 활동 · 연락을 별도 페이지로. "상단 메뉴로 이동하게"

문의 폼
서버 없이 폼을 받는 무료 서비스(예: Formspree) 연결

내용 파일 분리
활동 목록을 텍스트 파일로 빼고 파일만 고치면 반영

내 도메인
구입한 도메인을 Settings → Pages → Custom domain에 연결

모두 같은 방식

말하고 → 보고 → 커밋 · push. 새 기능을 넣기 전에 `Shift+Tab`으로 **계획(plan) 모드**에서 "어떻게 할 건지 먼저 보여줘"라고 하면 방향을 미리 잡을 수 있다.

더 읽을 곳

Claude Code 공식 문서 `code.claude.com/docs` (터미널 입문 안내 포함) · GitHub Pages 문서 `docs.github.com/pages`. 둘 다 Claude에게 "이 문서 읽고 설명해줘"라고 시켜도 된다.

4부 요약

1. **레퍼런스는 링크로 준다**: 템플릿(HTML5 UP · TemplateMo), 사례(One Page Love), 이미지(Unsplash), 글꼴(Pretendard). "이 느낌으로 새로"가 라이선스 걱정 없이 쉽다
2. **첫 프롬프트는 5요소로 길게**, 그 뒤는 한 줄씩. 형식(구성 · 느낌)과 제약(파일 하나 · 휴대폰 · 개인정보 없음)이 핵심
3. **말하기 → 보기 → 다시 말하기.** 브라우저 확인은 사람이 한다
4. **GitHub 올리기는 한 문장**: "git 저장소 만들고 아이디.github.io 공개 저장소로 push 해줘". 뒤에서 git · gh가 돈다
5. **저장소 이름 = 주소.** 아이디.github.io면 https://아이디.github.io
6. **Pages**: 자동으로 켜지거나 Settings → Pages → main / root. 1~10분
7. **수정 루프**: 고치고 → 확인 → 커밋 · push → 새로고침. 이력이 남고 되돌릴 수 있다

★ 한 문장으로

코드 대신 **요청 · 확인 · 승인**을 배웠다. 나머지는 Claude Code가 친다.

오늘의 핵심 정리

8차시에서 배운 것

1. **설치 3개**: Git · gh · Claude Code. 각 한 줄, 설치 후 새 창, `--version` 세 줄로 확인
2. **Claude Code = 손이 있는 Claude**: 내 폴더의 파일을 만들고 명령을 실행한다. 그래서 **권한 모드**가 있다
3. **권한 모드**: 자동(기본) · 수동 · 편집 자동 수락 · 계획. `Shift+Tab`. 승인 창은 도구 · 대상 · 되돌릴 수 있나를 읽는다
4. **명령은 둘만**: 막히면 `/help`, 멈추려면 `Esc`. 나머지는 `/`를 치면 보인다
5. **요청은 과제 · 형식 · 제약으로**. 한 번에 하나. "예쁘게"는 요청이 아니다
6. **막히면 4가지**: 에러 그대로 붙이기 · 되돌리기 · 쪼개기 · 다르게 말하기. 작업이 바뀌면 `/clear`
7. **GitHub**: 아이디가 주소. 공개 저장소는 전 세계 공개. 개인정보 · 비밀은 처음부터 넣지 않는다
8. **배포 루프**: 고치기 → 확인 → 커밋 · push → 새로고침. 실습실 PC가 초기화돼도 GitHub에 남는다

★ Big Picture

1차시 "**어떻게 물어보느냐**", 2차시 "**잘 물어본 것을 어떻게 저장하느냐**", 오늘 "**물어본 결과를 어떻게 세상에 내놓느냐**". 도구는 바뀌어도 습관은 같다.

집에서 이어하기 · 실습실 PC 정리

내 PC에서 이어서

1) 1부 명령으로 Git · gh · Claude Code 설치, `gh auth login`
2) 오늘 저장소 받아오기(아래 세 줄)
3) 첫 마디: "이 폴더가 어떤 상태인지 요약해줘"
4) 고치고 → 확인 → 커밋 · push
전문은 `prompts/10_home_setup.txt`

나가기 전 5줄 (공용 PC)

✓ Claude Code에서 `/logout`
✓ 터미널에서 `gh auth logout`
✓ 브라우저에서 `github.com` · `claude.ai` 로그아웃
✓ 브라우저 저장 비밀번호 · 자동완성 삭제
✓ 내 주소 `https://아이디.github.io`를 휴대폰에 메모

저장소 받아오기 (내 PC 터미널)

```
gh repo clone 아이디/아이디.github.io
cd 아이디.github.io
claude
```

이번 주에 해볼 것: 항목 하나 추가하고 push까지 혼자 한 바퀴 · "더 해볼 것" 중 하나를 계획 모드로 먼저 물어보기 · 옆사람 주소 열어 보기. 모두 개인 계정 · 가상 프로필로.

강의자료 전체 공개 저장소

전 차시 강의자료 한곳에

github.com/minercode625/military-ai-general-course

1–8차시 슬라이드 PDF · 실습 프롬프트 전문 · 실습 파일 (스킬 · 노트북 · 자료). 로그인 없이 받는다.

휴대폰 카메라로 열기

받는 법

1) 저장소 페이지 오른쪽 **Releases** → 차시별 `N차시_배포.zip`
2) 또는 터미널에서 `gh repo clone minercode625/military-ai-general-course`
3) 폴더 `1차시/`부터 `8차시/`까지, 각각 PDF와 `prompts/`

이렇게 쓰자

집에서 이어하기는 그 차시 `prompts/00_README.txt`부터 순서대로. 1 · 2차시 프롬프트는 업무 보고서에 바로 쓴다. 오늘 만든 홈페이지는 **내 계정**에, 이 저장소는 **강사 자료**.



Q & A

질문 있으신가요?